STUDENT RECORD MANAGEMENT SYSTEM

A modular Python-based Student Record Management System with profile search, grade evaluation, percentage calculation, and automated testing.

Course Name: Python Essentials

Student Name: Gupta Shourya Manohar

Registration Number: 26BCE10053

Institution: VIT Bhopal University

GitHub Repository:

1. Executive Summary & Introduction

Educational institutions handle large volumes of student records, requiring accurate maintenance of personal profile information alongside dynamic academic evaluations. Manual processing across unorganized spreadsheets often results in calculation discrepancies, record duplication, and delayed administrative operations.

The Student Record Management System is a lightweight, command-line interface (CLI) application developed in Python. Designed using strict modular programming principles, it decouples the data tier, business logic, user administration, interactive menu interface, and unit testing into distinct Python modules.

2. Problem Statement

Manual student record tracking faces several key operational limitations:

Calculation Discrepancies: Manual entry and computation of subject totals, percentages, and pass/fail statuses are highly prone to human error.
Inconsistent Data Formats: Inconsistent structuring of student records across multiple physical sheets leads to formatting issues and retrieval delays.
Lack of Direct Searching: Searching for specific student records in unindexed files or physical registers is slow and inefficient.

3. Scope of the System

In-Scope Capabilities

In-memory dataset handling using native Python dictionaries.
Direct lookup of detailed individual student profiles by Student ID.
Interactive registration of new student profiles with demographic metadata (Name, Age, State, Course, Semester, Hosteller/Day Scholar status).
Academic evaluation engine for 3 subjects (Math, Physics, CS) calculating total marks, overall percentages, and pass/fail statuses.
Modification of subject marks with automatic re-calculation of academic performance metrics.
Formatted multi-student summary table generator.
Automated unit validation suite utilizing assert statements.

Out-of-Scope Elements

External relational databases (SQL/NoSQL) or persistent storage modules.
Graphical User Interface (GUI) or web application interface.

4. Functional Requirements

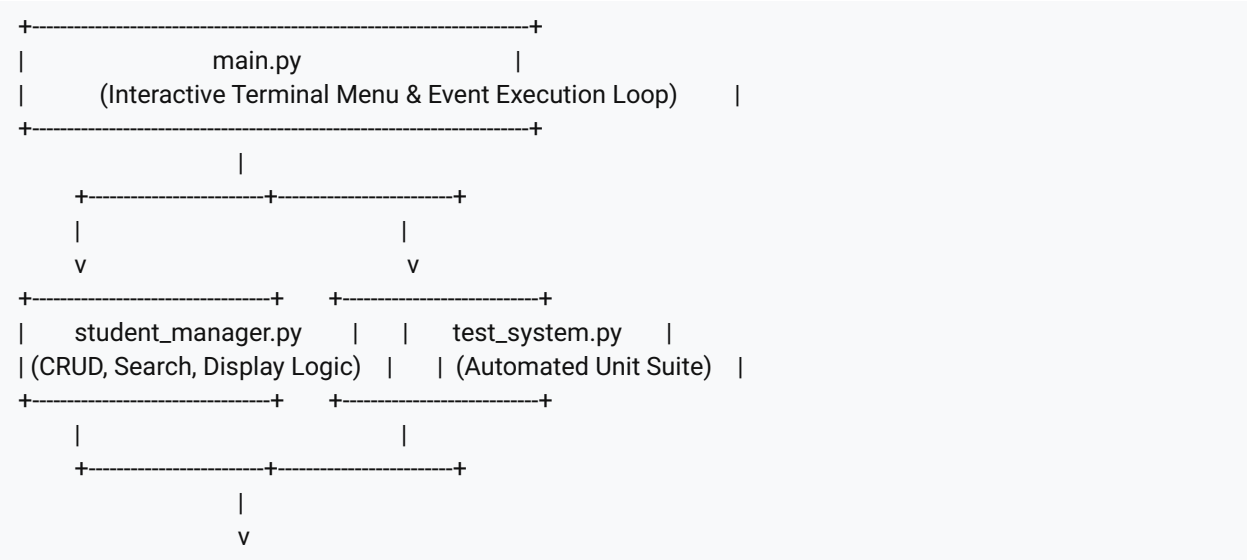
FR1 (Direct Record Lookup): The system shall allow users to search for any student profile using a case-insensitive Student ID (e.g., VIT001).
FR2 (Profile Registration): The system shall prompt for user demographic metadata and initial subject marks to create new records.
FR3 (Academic Evaluation): The system shall calculate total marks, overall percentage, and pass/fail status based on institutional passing criteria (>= 40 in each subject AND overall percentage >= 40.0%).
FR4 (Marks Modification): The system shall permit updates to existing subject marks and immediately trigger calculation refreshes.
FR5 (Summary Display): The system shall render a multi-column table displaying summary details across all registered students.

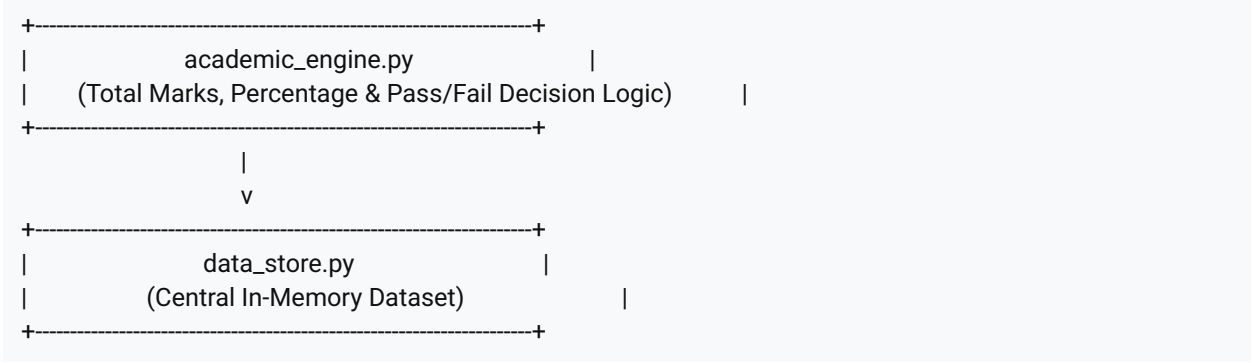
5. Non-Functional Requirements

Maintainability: Pure modular design separation across 5 core Python modules (main.py, student_manager.py, academic_engine.py, data_store.py, test_system.py).
Usability: Menu-driven CLI navigation with feedback and fallback options for invalid input choices.
Portability & Zero External Dependencies: Built strictly using core Python constructs without third-party external library requirements.

6. System Architecture & Modular Design

The system architecture decouples data management, logic processing, interface, and automated validation:





Module Responsibilities

1. **data_store.py:** Serves as the central repository holding student records structured as nested dictionaries.
2. **academic_engine.py:** Houses the core calculation function (calculate_academic_performance()) and input processing routines (prompt_marks_input()).
3. **student_manager.py:** Executes record search operations, formats summary tables, updates existing marks, and handles profile additions.
4. **main.py:** Provides the top-level menu, interprets user menu choices, and coordinates function calls across modules.
5. **test_system.py:** Executes unit validation checks on academic calculation logic.

7. Implementation & Code Structure

The academic calculation engine forms the functional core of the application:

```
def calculate_academic_performance(marks_dict):
    """
    Calculates total marks, overall percentage, and determines pass/fail status.
    Passing criteria: At least 40 marks in every subject AND an overall percentage >= 40.0%.
    """
    if not marks_dict:
        return 0, 0.0, "N/A"
    total = 0
    for subject in marks_dict:
        total += marks_dict[subject]
    percentage = round(total / len(marks_dict), 2)
    passed_all = True
    for subject in marks_dict:
        if marks_dict[subject] < 40:
            passed_all = False
            break
    if passed_all and percentage >= 40.0:
        result = "Pass"
    else:
```

```
        result = "Fail"
    return total, percentage, result
```

8. Verification & Testing Strategy

To ensure numerical and logical accuracy across performance evaluations, an automated unit test suite (test_system.py) was implemented:

```
from academic_engine import calculate_academic_performance
def run_tests():
    print("Running system validation tests...")
    # Test 1: Passing condition
    sample_marks_pass = {"Math": 80, "Physics": 75, "CS": 85}
    total, percentage, result = calculate_academic_performance(sample_marks_pass)
    assert total == 240, f"Expected 240, got {total}"
    assert percentage == 80.0, f"Expected 80.0, got {percentage}"
    assert result == "Pass", f"Expected Pass, got {result}"
    print("Test 1 (Pass Calculation): PASSED")
    # Test 2: Failing condition (One subject below 40)
    sample_marks_fail = {"Math": 35, "Physics": 75, "CS": 85}
    total, percentage, result = calculate_academic_performance(sample_marks_fail)
    assert result == "Fail", f"Expected Fail, got {result}"
    print("Test 2 (Fail Calculation): PASSED")
    print("\nAll validation tests completed successfully!")
if __name__ == "__main__":
    run_tests()
```

Execution Verification Output

```
Running system validation tests...
Test 1 (Pass Calculation): PASSED
Test 2 (Fail Calculation): PASSED
All validation tests completed successfully!
```

9. Conclusion & Future Enhancements

Conclusion

The Student Record Management System meets its administrative requirements. By adhering to modular design principles, the application decouples data representation, academic calculation logic, and interaction management, ensuring high code readability, maintainability, and reliable execution.

Future Enhancements

Persistent Storage Integration: Incorporating SQLite or JSON file persistence to retain records across sessions.
Graphical User Interface (GUI): Developing a Tkinter or Web-based dashboard interface.
Advanced Reporting: Exporting formal transcripts and performance reports as PDF or CSV files.